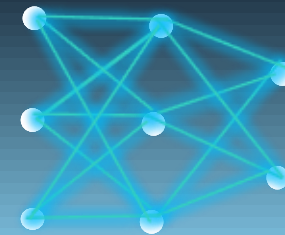




Deep Learning

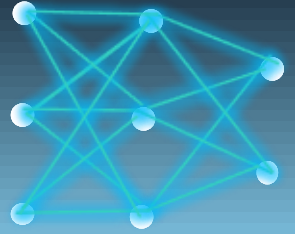
Dr. Shay Maymon



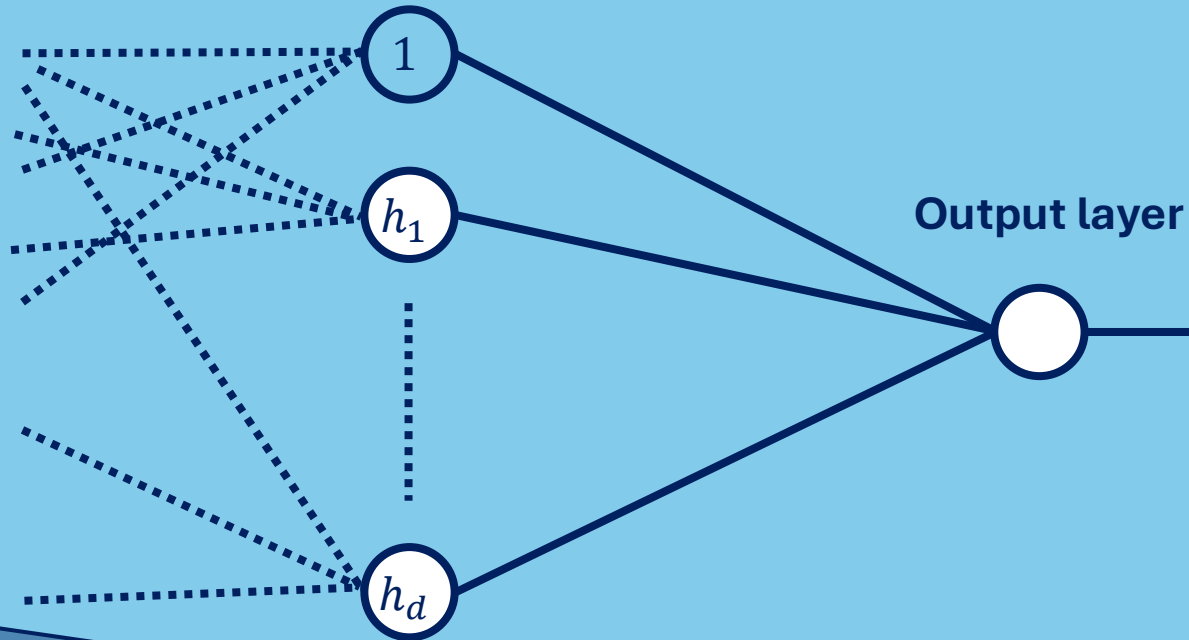
Overview of Today's Topics

- Deep Feedforward networks
 - ❖ Cost function and output units
 - ❖ Hidden Units

Linear units for Gaussian Output distribution

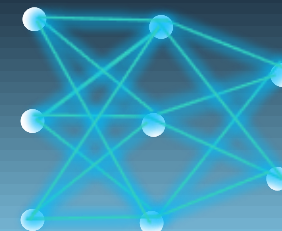


Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ i.i.d.



$$\hat{y} = \mathbf{w}^T \mathbf{h} + b$$
$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y; f(\mathbf{x}; \boldsymbol{\theta}), \sigma^2)$$

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^N (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2$$

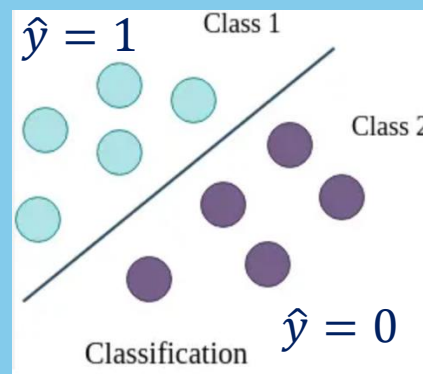
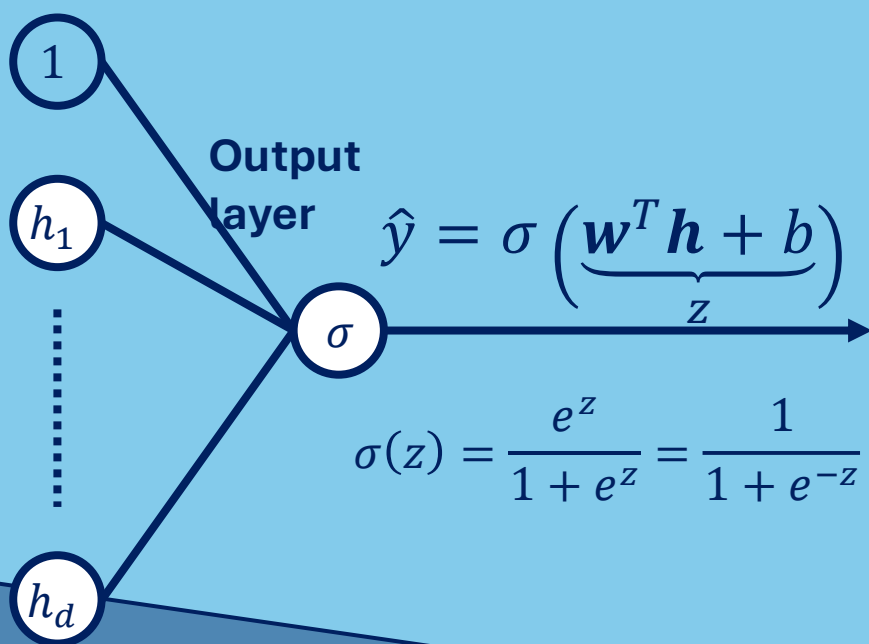


Sigmoid units for Bernoulli Distributions

- Binary classification

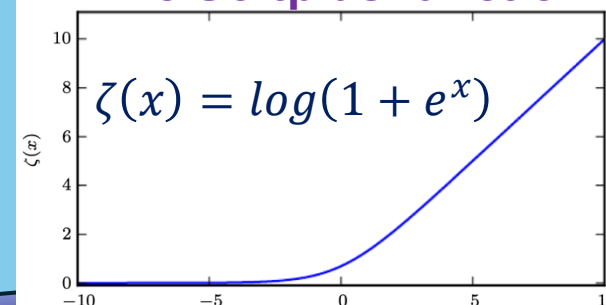
Training Data: $\{(x_n, y_n)_{n=1}^N\}$ i.i.d.

$$J(\theta) = - \sum_{n=1}^N y_n \log(\sigma(z_n)) + (1 - y_n) \log(1 - \sigma(z_n))$$



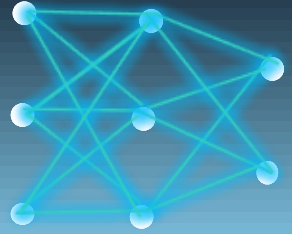
$$J(\theta) = \sum_{n=1}^N \zeta((1 - 2y_n)z_n)$$

The Softplus function

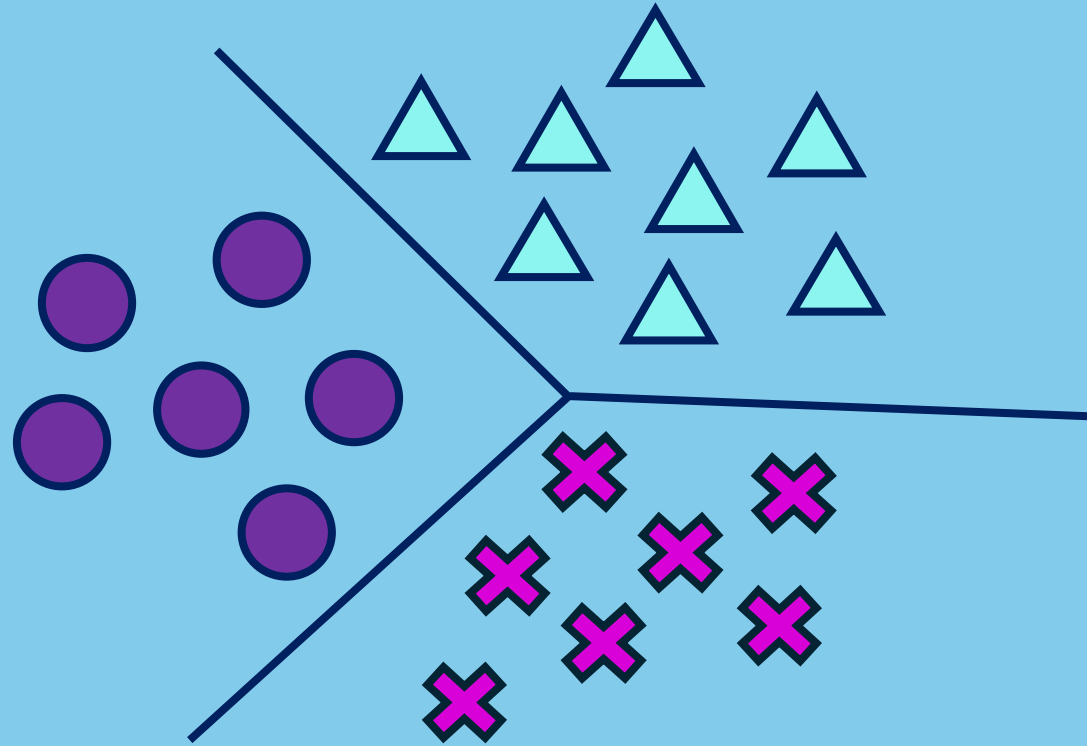


- Saturation thus occurs only when the model already has the right answer
- Gradient-based learning can act to quickly correct a mistakenz

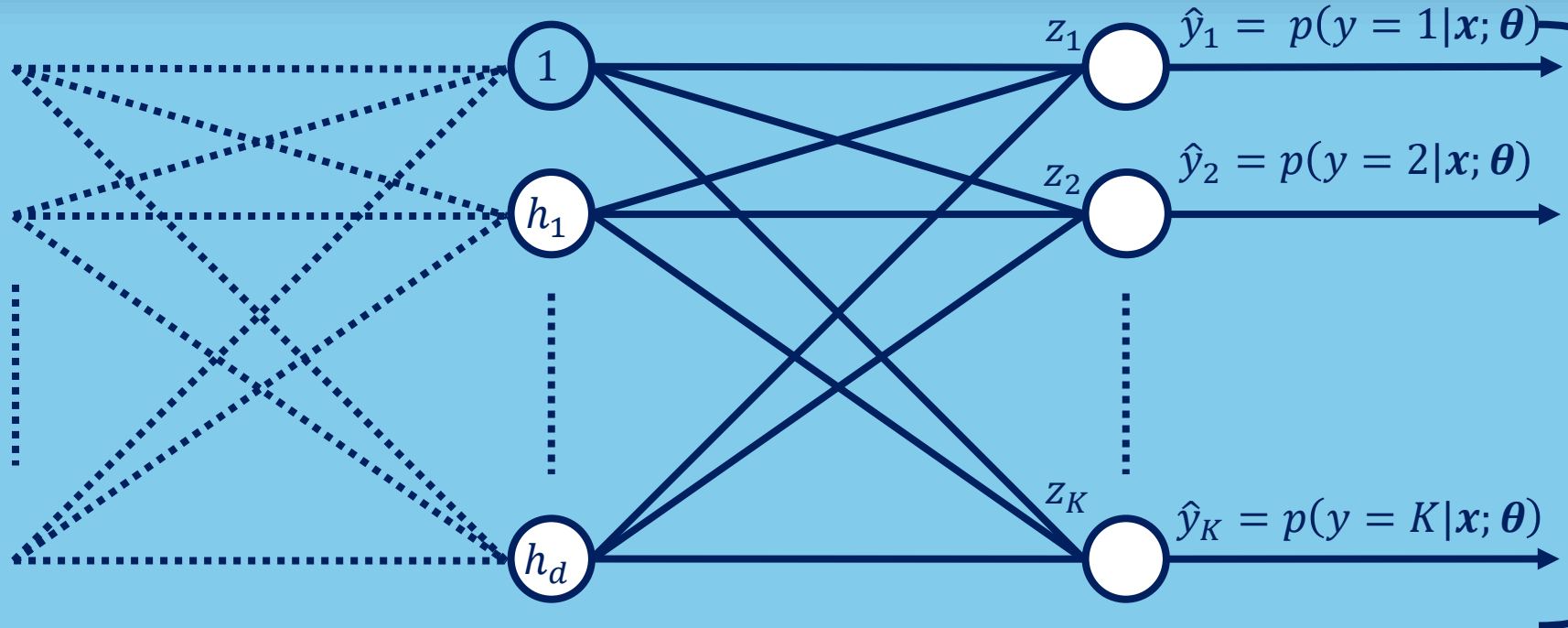
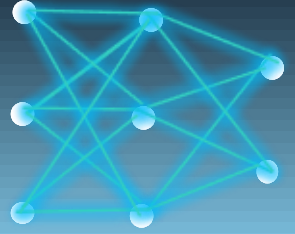
Multiclass Classification



Each input is assigned to one of K mutually exclusive classes



Multiclass Classification



• Any time we wish to **represent** a **probability distribution** over a **discrete variable** with K **possible values**, we may use the **softmax function**

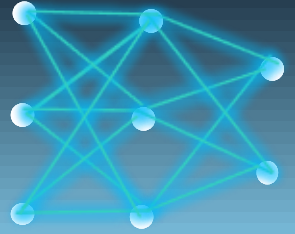
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}^T \mathbf{h} + \mathbf{b}$$

$$z_i = \log(\tilde{p}(y = i | \mathbf{x}; \boldsymbol{\theta}))$$

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

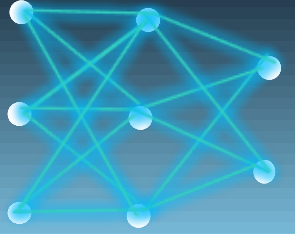
Invariance of softmax under Translation



- The **softmax output** is **invariant** to **adding the same scalar** to all of its inputs

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Invariance of softmax under Translation

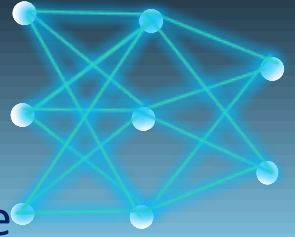


- The **softmax output** is **invariant** to **adding the same scalar** to all of its inputs

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i - c)}{\sum_j \exp(z_j - c)}$$

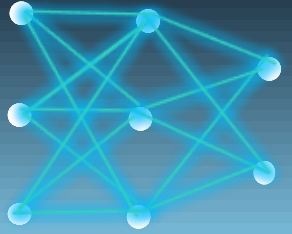
$$\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - \max_j(z_j))$$

Saturation of the Softmax function



- Like the **sigmoid**, the **softmax** activation can **saturate**. In the case of the **softmax**, there are **multiple output values**. These **output values** can **saturate** when the **differences between input values** become extreme
- When the **softmax** saturates, many **cost functions** based on the **softmax** also saturate, unless they are able to invert the saturating activating function. Many **objective functions** other than the log-likelihood **do not work well** with the **softmax function**. In particular, **squared error** is a **poor loss function** for **softmax** units and **can fail to train** the model to change its output, even when the model makes highly confident incorrect predictions
- An **output** $\text{softmax}(z)$ **saturates to 1** when the corresponding **input is maximal** ($z_i = \max_j (z_j)$) and z_i is **much greater** than **all the other inputs**. The **output** can also **saturate to 0** when z_i is not maximal and **the maximum is much greater**

Derivative of Softmax

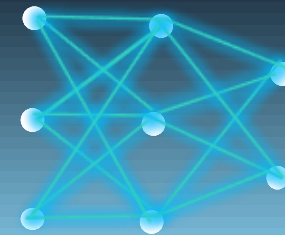


$$p_i = p(y = i | \mathbf{x}; \boldsymbol{\theta}) = \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)}$$

$$\begin{aligned} \frac{\partial p_i}{\partial z_i} &= \frac{\exp(z_i) \left(\sum_{k=1}^K \exp(z_k) \right) - \left(\exp(z_i) \right)^2}{\left(\sum_{k=1}^K \exp(z_k) \right)^2} = \frac{\exp(z_i) \left(\sum_{k=1}^K \exp(z_k) - \exp(z_i) \right)}{\left(\sum_{k=1}^K \exp(z_k) \right)^2} \\ &= \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \cdot \frac{\sum_{k=1}^K \exp(z_k) - \exp(z_i)}{\sum_{k=1}^K \exp(z_k)} = \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \cdot \left(1 - \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \right) = p_i(1 - p_i) \end{aligned}$$

$$\frac{\partial p_i}{\partial z_j} = \frac{0 - \exp(z_j) \exp(z_i)}{\left(\sum_{k=1}^K \exp(z_k) \right)^2} = - \left(\frac{\exp(z_j)}{\sum_{k=1}^K \exp(z_k)} \right) \left(\frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \right) = -p_i p_j, j \neq i$$

$$\frac{\partial p_i}{\partial z_j} = p_i (\delta_{ij} - p_j)$$

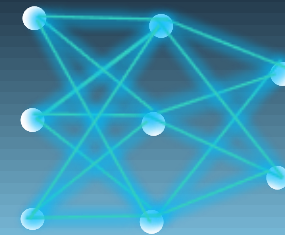


Multiclass Classification – Loss Function

Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ **i.i.d.**

$$L(\boldsymbol{\theta}) = p(\mathbf{y}|\{\mathbf{x}_1, \dots, \mathbf{x}_N\}; \boldsymbol{\theta}) = \prod_{n=1}^N p(y_n|\mathbf{x}_n; \boldsymbol{\theta})$$

$$p(y_n|\mathbf{x}_n; \boldsymbol{\theta}) = \prod_{k=1}^K (p(y_n = k|\mathbf{x}_n; \boldsymbol{\theta}))^{I(y_n=k)}$$

Multiclass Classification – Loss Function

Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ i.i.d.

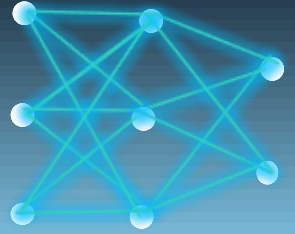
$$L(\boldsymbol{\theta}) = p(\mathbf{y}|\{\mathbf{x}_1, \dots, \mathbf{x}_N\}; \boldsymbol{\theta}) = \prod_{n=1}^N \prod_{k=1}^K (p(y_n = k | \mathbf{x}_n; \boldsymbol{\theta}))^{I(y_n=k)}$$

$$J(\boldsymbol{\theta}) = -\log L(\boldsymbol{\theta}) = -\log \prod_{n=1}^N \prod_{k=1}^K (p(y_n = k | \mathbf{x}_n; \boldsymbol{\theta}))^{I(y_n=k)}$$

$$J(\boldsymbol{\theta}) = -\sum_{n=1}^N \sum_{k=1}^K I(y_n = k) \log (p(y_n = k | \mathbf{x}_n; \boldsymbol{\theta}))$$

$$J(\boldsymbol{\theta}) = \sum_{n=1}^N \underbrace{\sum_{k=1}^K -y_{nk} \log (\hat{y}_{nk})}_{-\log(\hat{y}_{nc})} = \sum_{n=1}^N \sum_{k=1}^K -y_{nk} \left(z_{nk} - \log \left(\sum_{j=1}^K \exp(z_{nj}) \right) \right)$$

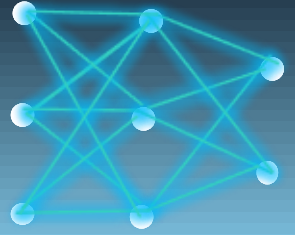
Multiclass Classification – Loss Function



Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ **i.i.d.**

$$J(\boldsymbol{\theta}) = \sum_{n=1}^N -\log(\hat{y}_{nc}) = \sum_{n=1}^N - \left(z_{nc} - \log \left(\sum_{j=1}^K \exp(z_{nj}) \right) \right)$$

Softmax Units for Multinoulli Output distributions



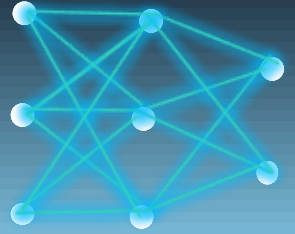
- As with the logistic sigmoid, **the use of the exp function works very well when training the softmax to output a target value y using maximum log-likelihood**
- The **log** in the **log-likelihood** can **undo** the **exp** of the softmax:

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

$$\log \text{softmax}(\mathbf{z})_i = z_i - \log \sum_j \exp(z_j)$$

- The **first term cannot saturate - learning can proceed**, even if the contribution of z_i to the second term becomes very small

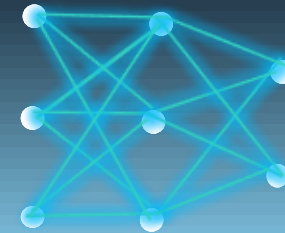
Softmax Units for Multinoulli Output distributions



$$\log \text{softmax}(\mathbf{z})_c = z_c - \log \sum_j \exp(z_j)$$

When maximizing the log-likelihood, the first term encourages z_c to be pushed up, while the second term encourages all of $\mathbf{z}$ to be pushed down

Softmax Units for Multinoulli Output distributions



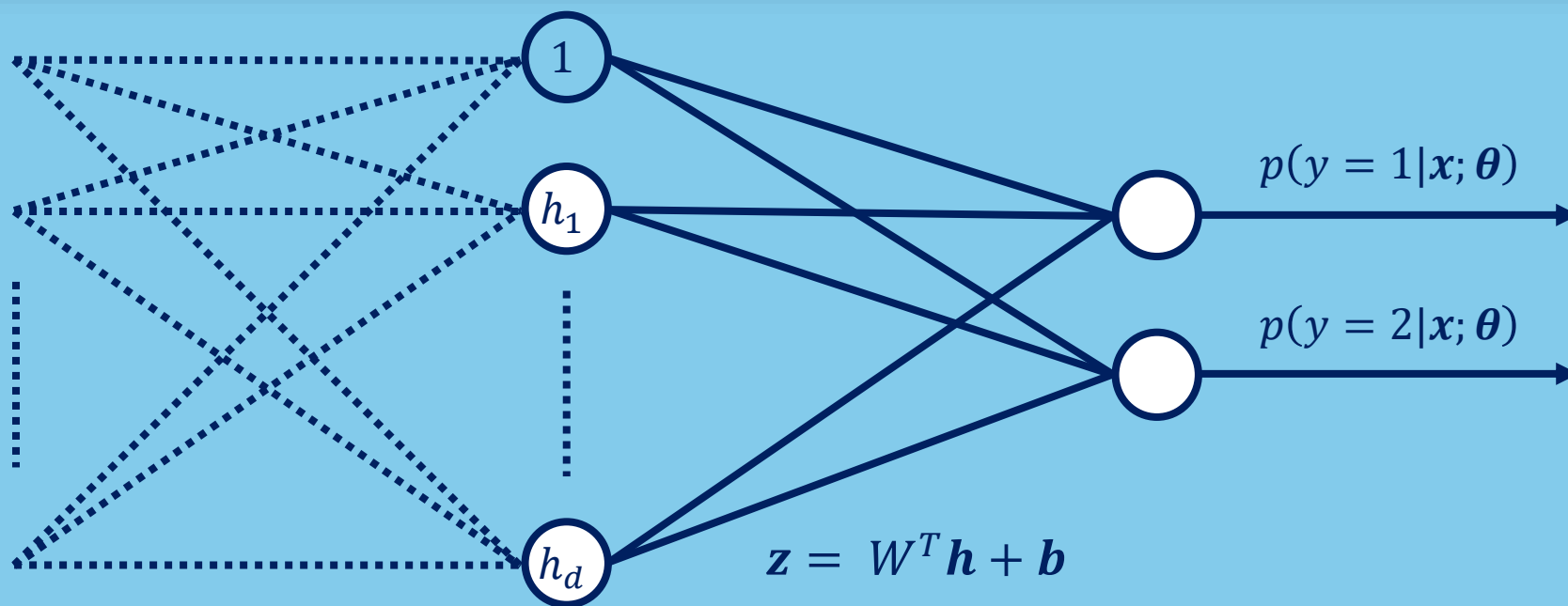
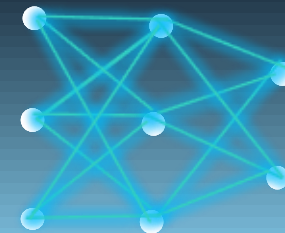
$$\log\left(\sum_j \exp(z_j)\right) \approx \max_j(z_j)$$

$$-\log \text{softmax}(\mathbf{z})_c \approx \max_j(z_j) - z_c$$

- If the **correct answer already has the largest input to the softmax**, then the $-z_c$ term and the $\log(\sum_j \exp(z_j)) \approx \max_j(z_j) = z_c$ terms will roughly **cancel**. This example will then **contribute little to the overall training cost**, which will be dominated by other examples that are not yet correctly classified
- The **negative log-likelihood cost function** always strongly **penalizes the most active incorrect prediction**

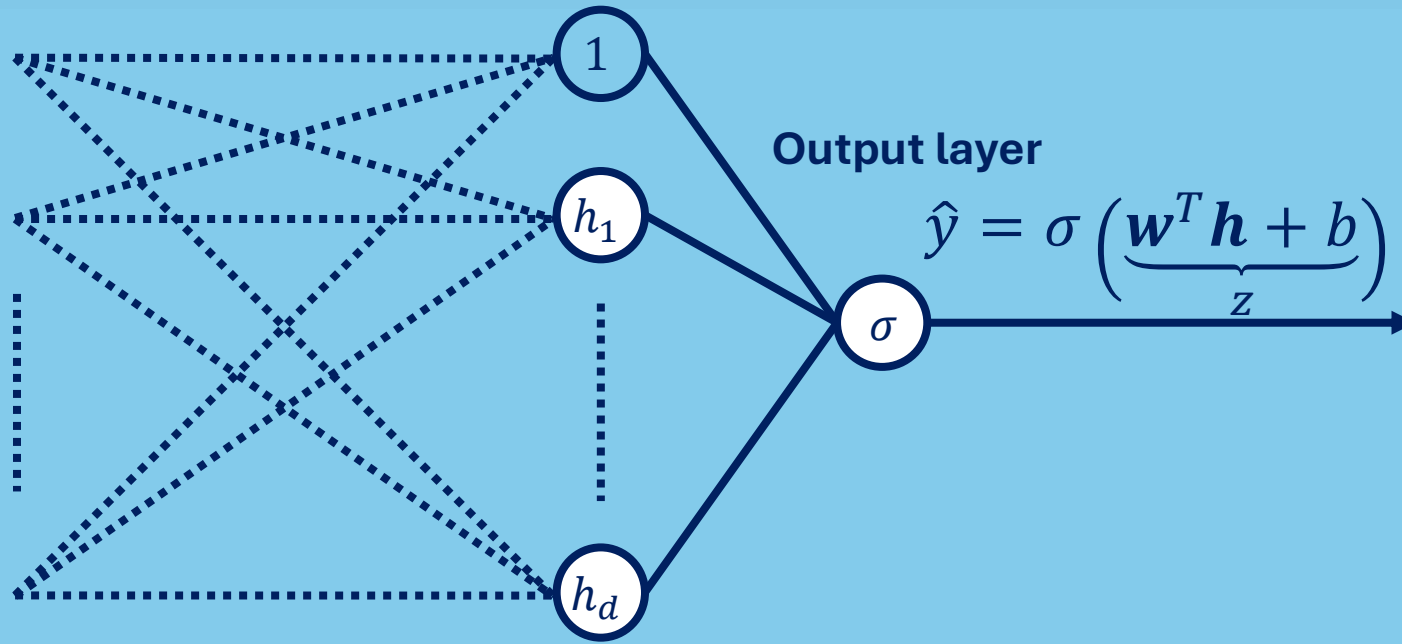
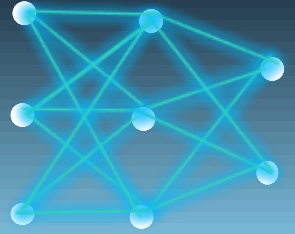
$$J(\boldsymbol{\theta}) = \sum_{n=1}^N -\log(\hat{y}_{nc}) = \sum_{n=1}^N -\left(z_{nc} - \log\left(\sum_{j=1}^K \exp(z_{nj})\right) \right)$$

Binary Classification

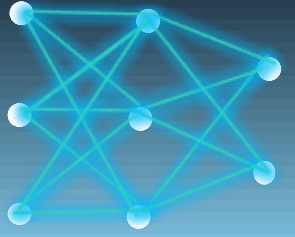


$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$
$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

Binary Classification

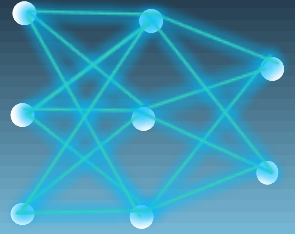


Equivalence to Sigmoid (K=2)



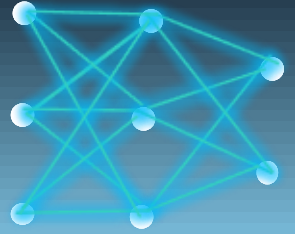
$$\begin{aligned}\hat{y} = f(\mathbf{x}; \boldsymbol{\theta}) &= \begin{bmatrix} p(y = 1 | \mathbf{x}; \boldsymbol{\theta}) \\ p(y = 2 | \mathbf{x}; \boldsymbol{\theta}) \end{bmatrix} = \begin{bmatrix} \frac{\exp(z_1)}{\sum_{j=1}^2 \exp(z_j)} \\ \frac{\exp(z_2)}{\sum_{j=1}^2 \exp(z_j)} \end{bmatrix} = \begin{bmatrix} \frac{\exp(\tilde{\mathbf{w}}_1^T \tilde{\mathbf{h}})}{\exp(\tilde{\mathbf{w}}_1^T \tilde{\mathbf{h}}) + \exp(\tilde{\mathbf{w}}_2^T \tilde{\mathbf{h}})} \\ \frac{\exp(\tilde{\mathbf{w}}_2^T \tilde{\mathbf{h}})}{\exp(\tilde{\mathbf{w}}_1^T \tilde{\mathbf{h}}) + \exp(\tilde{\mathbf{w}}_2^T \tilde{\mathbf{h}})} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\exp((\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}})}{\exp((\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}}) + \exp((\tilde{\mathbf{w}}_2 - \tilde{\mathbf{w}}_1)^T \tilde{\mathbf{h}})} \\ \frac{\exp((\tilde{\mathbf{w}}_2 - \tilde{\mathbf{w}}_1)^T \tilde{\mathbf{h}})}{\exp((\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}}) + \exp((\tilde{\mathbf{w}}_2 - \tilde{\mathbf{w}}_1)^T \tilde{\mathbf{h}})} \end{bmatrix} = \begin{bmatrix} \frac{1}{1 + \exp((\tilde{\mathbf{w}}_2 - \tilde{\mathbf{w}}_1)^T \tilde{\mathbf{h}})} \\ \frac{\exp((\tilde{\mathbf{w}}_2 - \tilde{\mathbf{w}}_1)^T \tilde{\mathbf{h}})}{1 + \exp((\tilde{\mathbf{w}}_2 - \tilde{\mathbf{w}}_1)^T \tilde{\mathbf{h}})} \end{bmatrix} \\ &= \begin{bmatrix} \frac{1}{1 + \exp(-(\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}})} \\ \frac{\exp(-(\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}})}{1 + \exp(-(\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}})} \end{bmatrix} = \begin{bmatrix} \sigma((\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}}) \\ 1 - \sigma((\tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2)^T \tilde{\mathbf{h}}) \end{bmatrix}\end{aligned}$$

Output-unit Activation and Cost-function



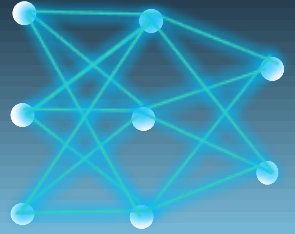
- There is a **natural choice** of both **output-unit activation function** and **matching error function** according to the type of problem being solved
 - **Regression** – **Linear outputs** & **sum-of-squares error**
 - **Binary Classification** – **Logistic sigmoid outputs** & **cross-entropy error function**
 - **Multi-class classification** – **Softmax outputs** & **multi-class cross-entropy error function**
- This procedure is **quite general**, and by **considering other forms of conditional distribution**, we can **derive the associated error functions** as the **corresponding negative log-likelihood**

The Design of hidden Units



- The **design of hidden units** is an extremely **active area of research** and **does not yet have** many **definitive guiding theoretical principles**
- It is usually **impossible to predict in advance** which will **work best**
- Some of the hidden units included in this list are not actually **differentiable** at all input points. This may seem like it **invalidates them for use** with a **gradient based learning algorithm**. **In practice, gradient descent still performs well enough** for these models to be used for machine learning tasks

The hidden Units

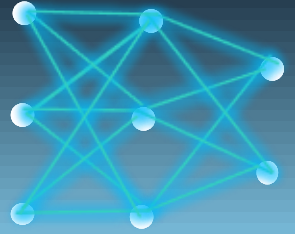


- **Most hidden units** can be described as accepting a vector of inputs x , computing an **affine transformation** and then **applying an elementwise nonlinear function** $g()$

$$g(W^T h + b)$$

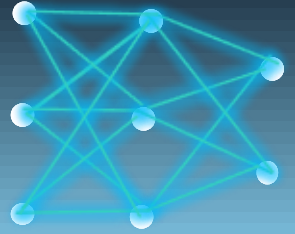
- **Most hidden units** are **distinguished** from each other only by **the choice of the form of the activation function** $g(z)$
- In most cases, **all the hidden units** in a network will be given the **same activation function**, although **in principle** there is no reason why **different choices** could not be applied in different parts of the network

The Simplest Option for a Hidden Unit



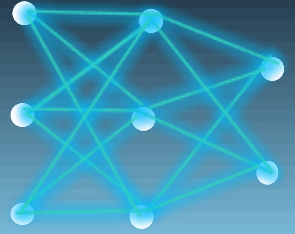
- The identity function $g(z) = z$
- The network's **representational capability** is no greater than that of a **single linear layer**

The Simplest Option for a Hidden Unit



- If the **number of hidden units** is **smaller** than the minimum between the **number of input** or **output** units, then the transformations are **not the most general possible linear transformation** because **information is lost** in the **dimensionality reduction** at the hidden units

The identity function



Consider a network with M **inputs**, K **hidden units**, and N **outputs**, and where **all activation functions are linear (assume no bias terms)**

Such a network has $K(M + N)$ **parameters**, whereas a linear transformation of inputs directly to outputs would have MN **parameters**

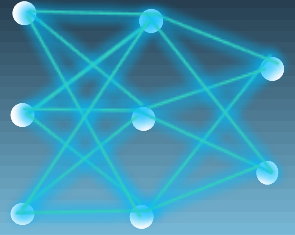
If M is small enough relative to N and K , this **may** lead to a two-layer linear network having **fewer parameters** than the direct linear mapping. It comes at the cost of constraining the linear transformation to be **low rank**

Such **bottleneck networks** of linear units corresponds to a standard **data analysis technique** called **principal component analysis**

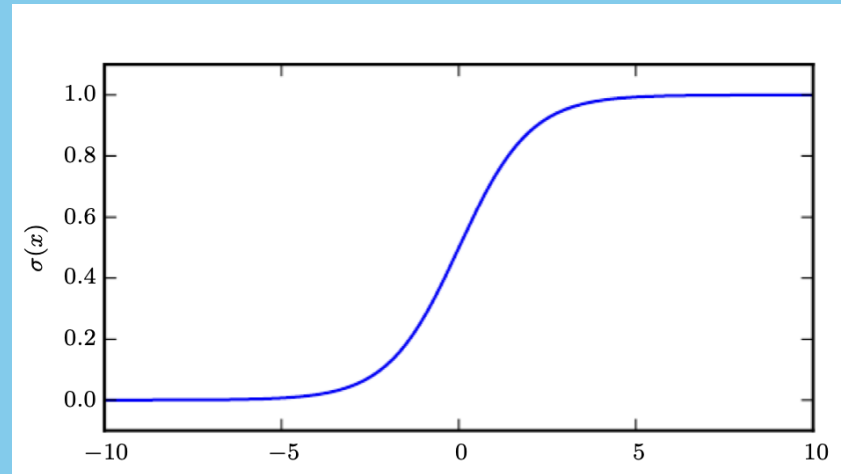
In general, there is **limited interest in using multilayer networks of linear units** since the **overall function** computed by such a network **is still linear**

Logistic Sigmoid

- A simple, nonlinear differentiable function is the **logistic sigmoid**

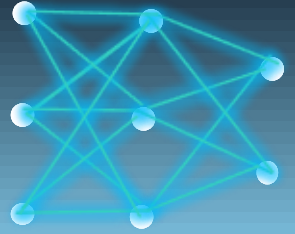


$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



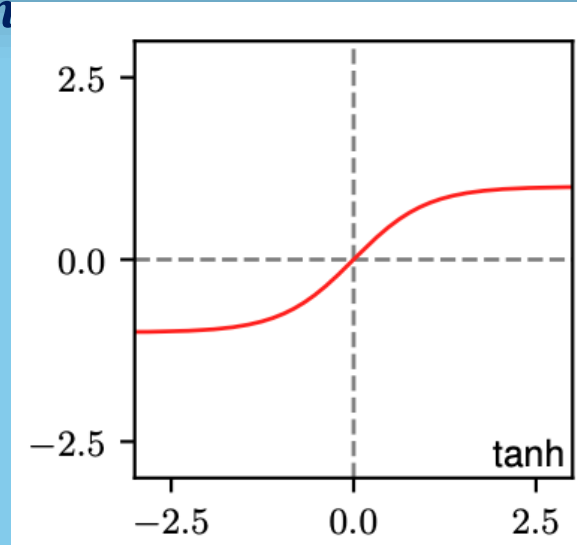
- This **activation function** was **widely used** in the **early years** of work on **multilayer neural networks** and was **partly inspired** by studies of the **properties of biological neurons**

Hyperbolic Tangent



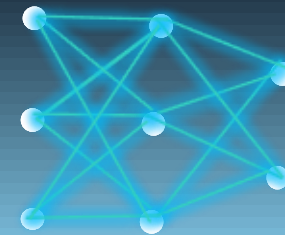
- A closely related function is *tanh*

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

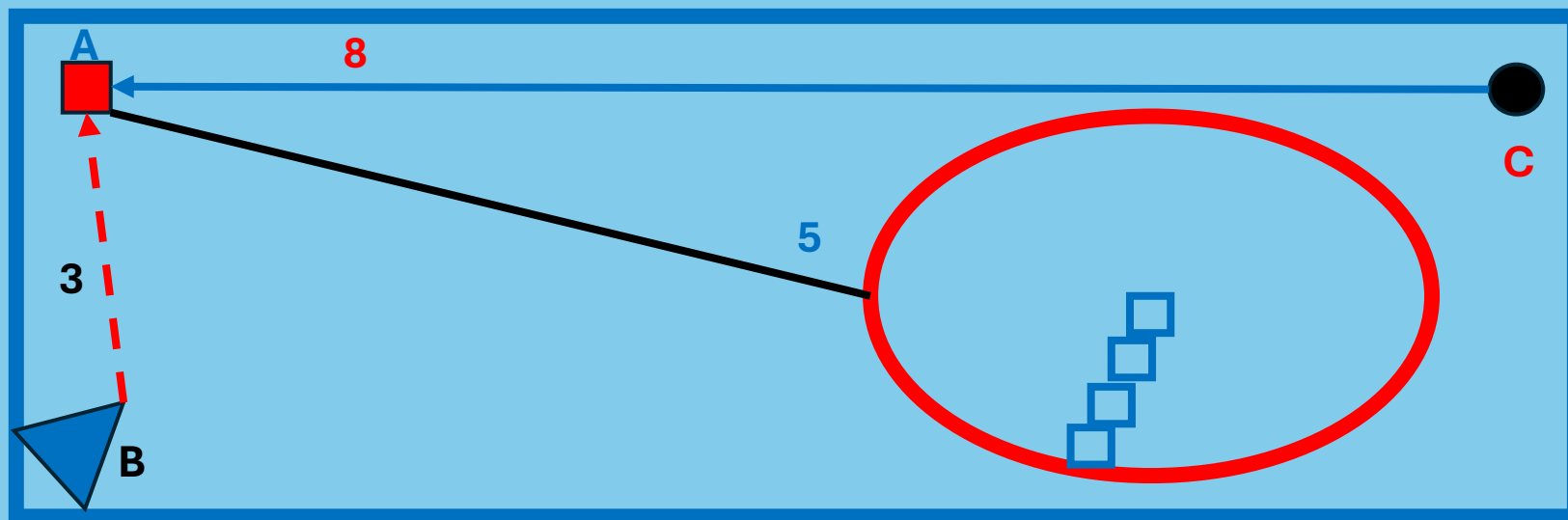
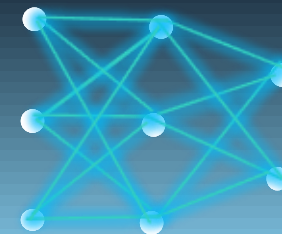


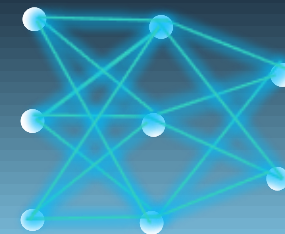
- For any network with **logistic-sigmoid hidden-unit activation functions**, there is an **equivalent** network with *tanh* activation functions

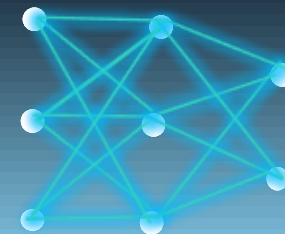
$$\tanh(\mathbf{z}) = 2\sigma(2\mathbf{z}) - 1$$

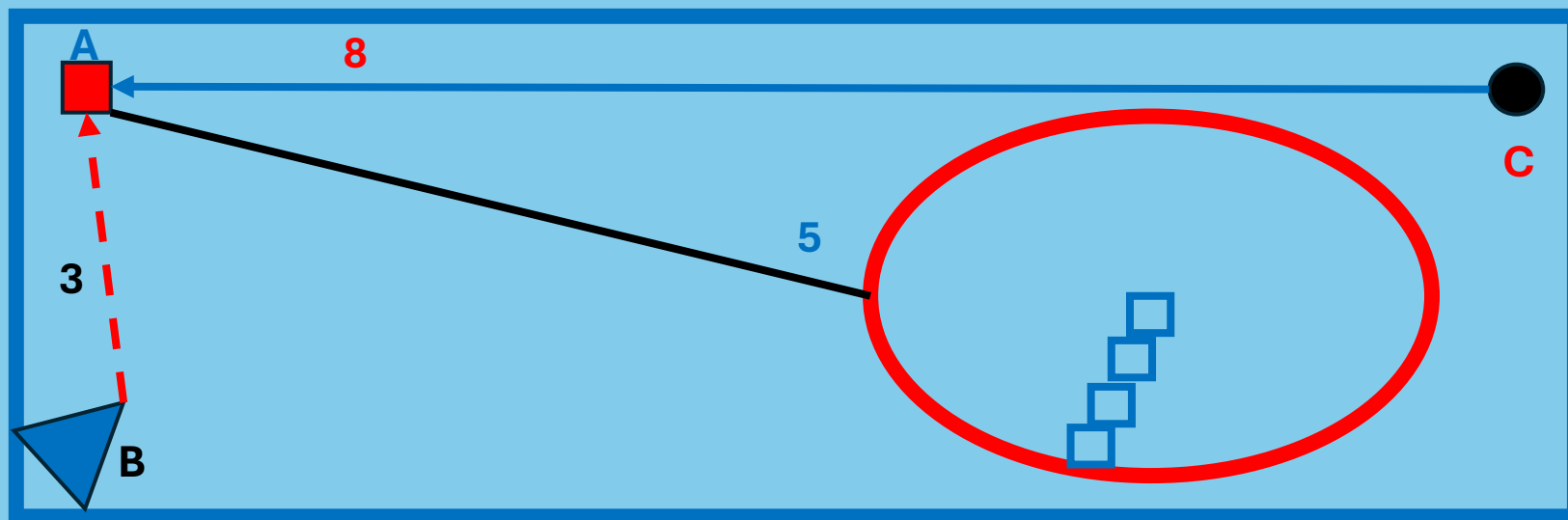
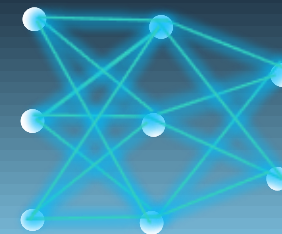


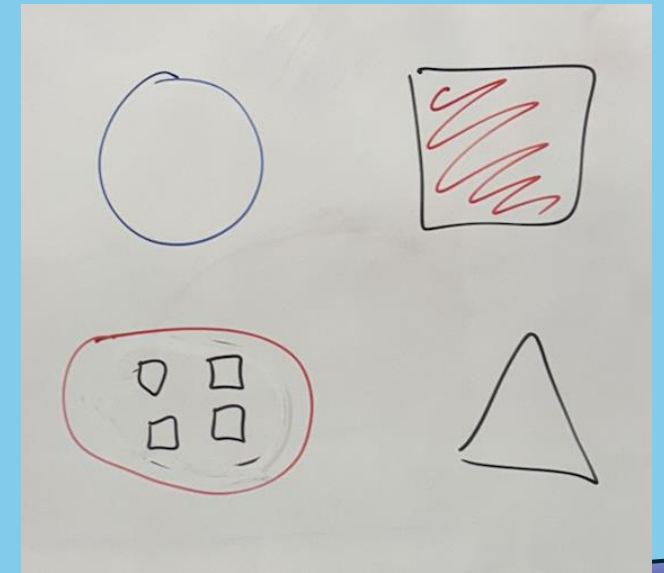
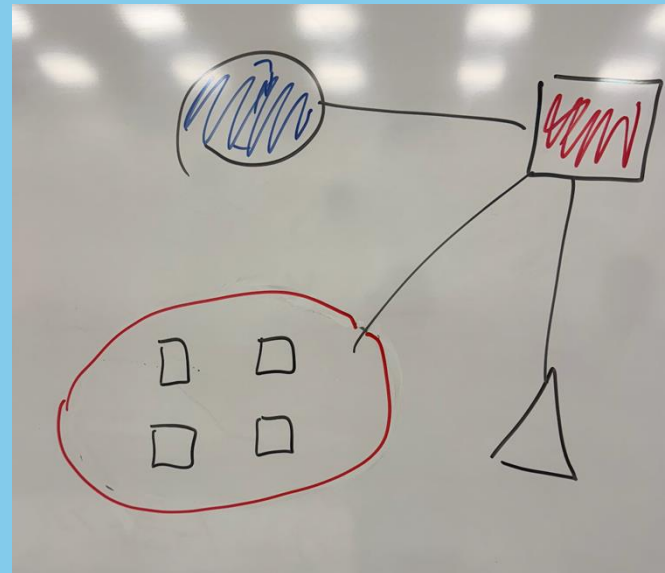
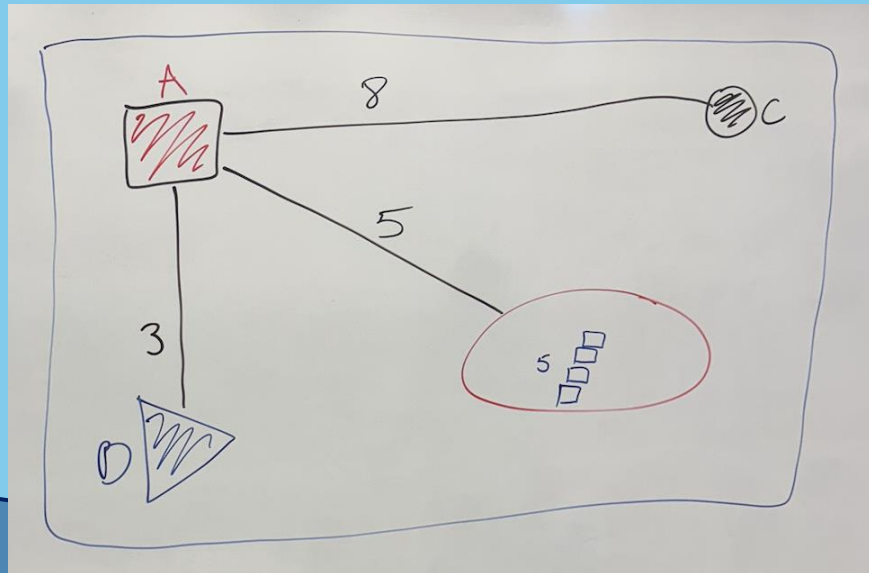
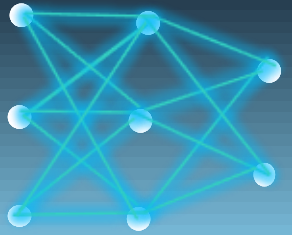
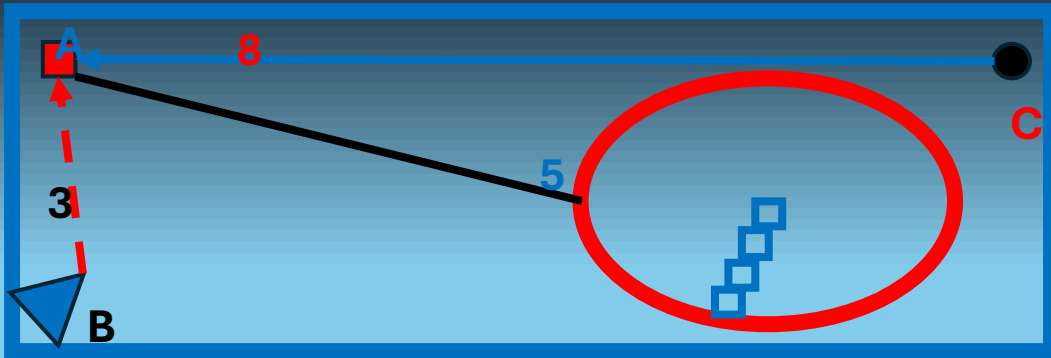
3 Volunteers?



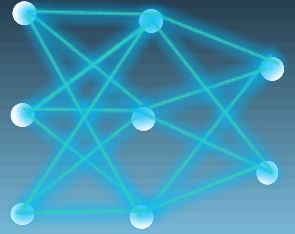






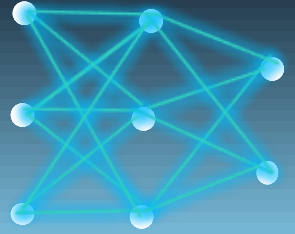


Logistic Sigmoid and Hyperbolic Tangent



- **Sigmoidal** units **saturate** across most of their domain – they **saturate to a high value when z is very positive, saturate to a low value when z is very negative**, and are only **strongly sensitive to their input when z is near 0**
- The widespread **saturation** of **sigmoidal** units can make **gradient-based learning** very **difficult**. For this reason, **their use as hidden units in feedforward networks is now discouraged**. We will discuss this ‘**vanishing gradients**’ issue later but for the moment, we note that it will generally be better to **use activation functions with non-zero gradients**
- Their use as **output units** is **compatible** with the use of **gradient-based learning** when an appropriate **cost function can undo the saturation** of the **sigmoid in the output layer**

Logistic Sigmoid and Hyperbolic Tangent

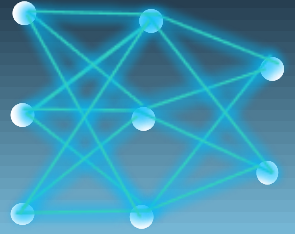


- When a **sigmoidal activation function** must be used, the **hyperbolic tangent** activation function typically **performs better** than the **logistic sigmoid**
- It **resembles the identity function more closely near 0**, and therefore **training a deep neural network with *tanh* resembles training a linear model so long as the activations of the network can be kept small**. This makes **training *tanh* network easier**

$$\hat{y} = \mathbf{w}^T \tanh(U^T \tanh(V^T \mathbf{x})) \approx \mathbf{w}^T U^T V^T \mathbf{x}$$

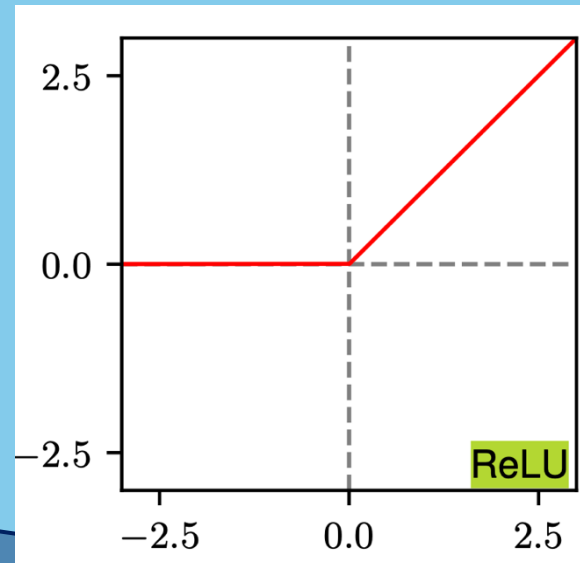
- **Sigmoidal activation functions are more common in settings other than feedforward network. Recurrent networks, probabilistic models, and some autoencoders have additional requirements that rule out the use of piecewise linear activation functions and make sigmoidal units more appealing despite the drawbacks of saturation**

Rectified Linear Units



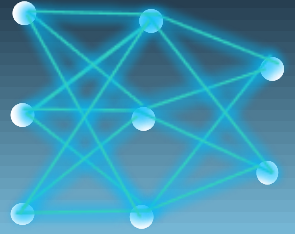
- A **simpler** choice of **activation function** is the **Rectified Linear Unit (ReLU)**
- **Rectified Linear Units** are **easy to optimize** because they are so **similar to linear units**
- The **derivatives** through a **rectified linear unit** remain **large** whenever the **unit is active**

$$h(x) = \max(0, x)$$



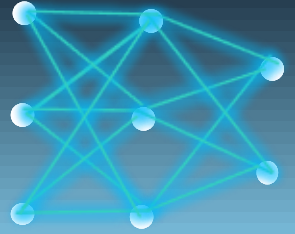
The derivative is not defined when $x=0$, but in practice this can be safely ignored

Rectified Linear Units – Parameters Initialization



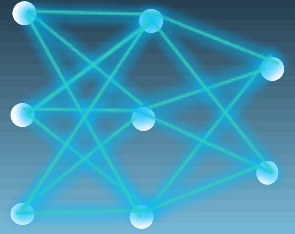
- When **initializing** the **parameters** of the **affine transformation**, it can be a **good practice** to set all elements of $\mathbf{b}$ to a **small, positive value**, such as 0.1. This makes it very likely that the **rectified linear units** will be **initially active** for most inputs in the training set and **allow the derivatives to pass through**

Rectified Linear Units – Drawbacks



- One **drawback** to **ReLU** is that they **cannot learn** via **gradient based methods** on **examples for which their activation is zero**. A **variety of generalizations** of **ReLUs** **guarantee** that they **receive gradient everywhere**
- Several **generalizations** of **ReLU** **exist**. Most of these generalizations **perform comparably** to **ReLUs** and **occasionally perform better**

A Leaky ReLU

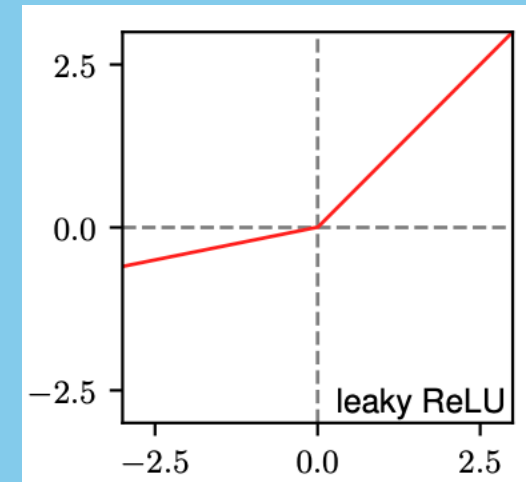


- Although the **ReLU** has a **non-zero gradient** for **positive input values**, this is **not the case** for **negative inputs** - **some hidden units receive no 'error signal' during training**
- A **modification** of ReLU that **seeks to avoid** this issue is called a **leaky ReLU** and is defined by

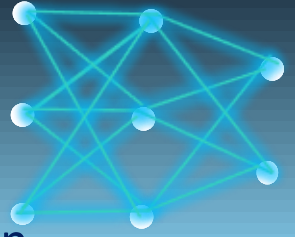
$$g(z) = \max(0, z) + \alpha \min(0, z), \quad 0 < \alpha < 1$$

- **Unlike ReLU**, this has a **nonzero gradient** for input values $a < 0$, which ensures that **there is a signal to drive training**

A Leaky ReLU fixes α to a small value like 0.01



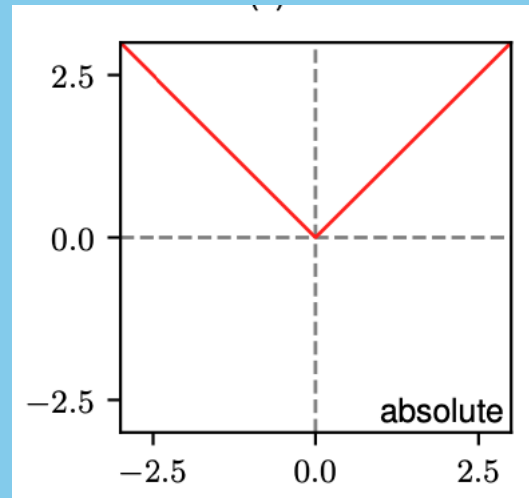
Absolute Value Rectification



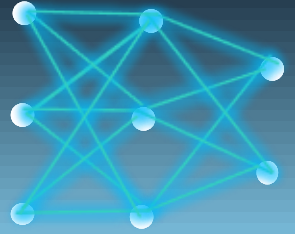
- Absolute value rectification fixes $\alpha=-1$ to obtain $g(z) = |z|$. It is used for object recognition from images

$$g(z) = \max(0, z) + \alpha \min(0, z)$$

$$g(z) = |z|$$



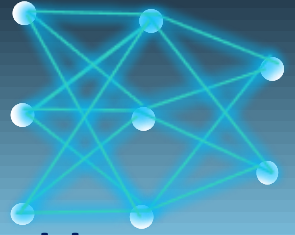
Parameteric ReLU



- **Parametric ReLU (PReLU)** treats α_i as a **learnable parameter**
- It allows **each hidden unit** to have its **own value** α_i , which can be **learned during network training** by evaluating gradients with respect to the α_i along with the gradients with respect to the weights and biases

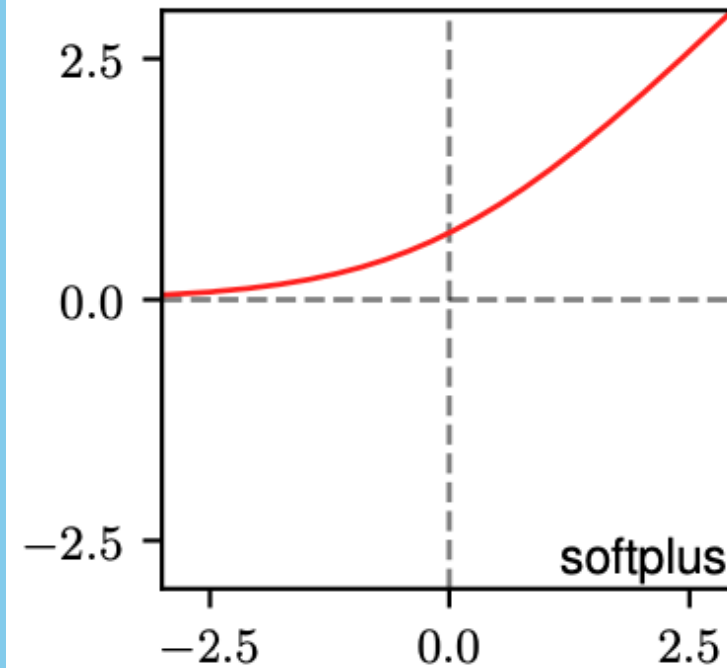
$$g(\mathbf{z}, \boldsymbol{\alpha})_i = \max(0, z_i) + \alpha_i \min(0, z_i)$$

The Softplus Activation Function

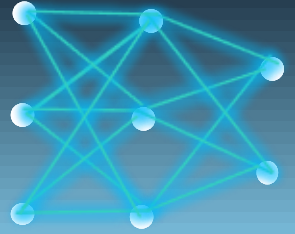


- For $a \gg 1$, we have $h(a) \approx a$ and so **the gradient remains non-zero** even when **the input to the activation function is large and positive**, thereby **helping to alleviate the vanishing gradients problem**

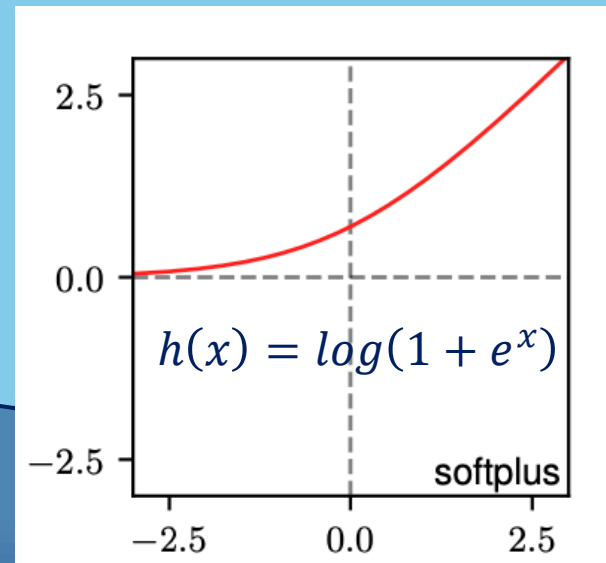
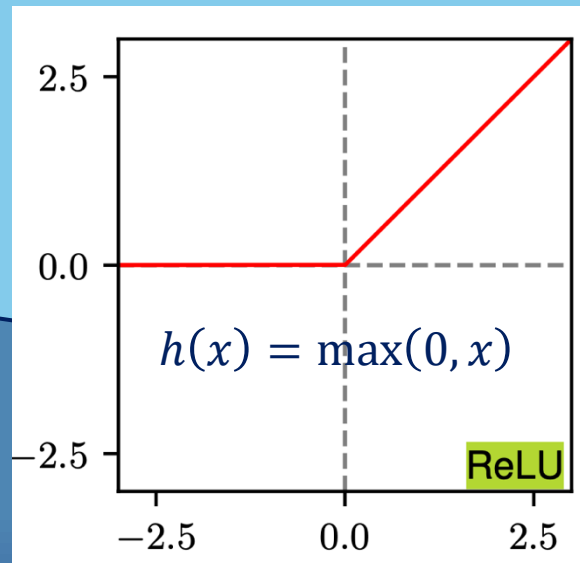
$$h(x) = \log(1 + e^x)$$



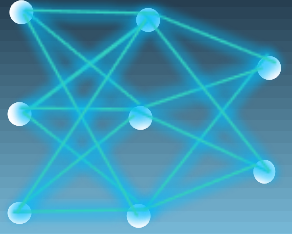
Softplus – Soft ReLU



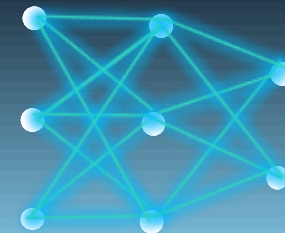
- The **softplus function** can be viewed as a **smoothed version of ReLU** and is therefore also sometimes called **soft ReLU**
- One might expect it to have an **advantage over the rectifier** due to being **differentiable everywhere** or due to **saturating less completely**, but **empirically it does not**
- The **use of softplus** is **generally discouraged**



Rectified Linear Units and Their Generalizations

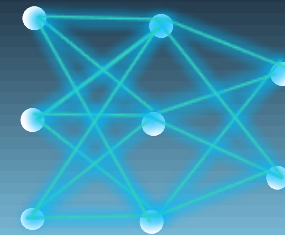


- ReLUs and all of these generalizations of them are **based on the principle** that models are **easier to optimize** if their **behavior is closer to linear**
- Empirically, ReLU is one of the **best-performing activation functions**, and it is in widespread use
- The introduction of ReLU gave a **big improvement** in **training efficiency over previous sigmoidal activation functions**
- It is **computationally cheap to evaluate**
- Many **practical applications simply use ReLU units as the default** unless the **goal is explicitly to explore the effects of different choices of activation function**



Key Takeaways

- Most modern neural networks are **trained using maximum likelihood**
- Functions that **saturate** make the **gradient** become **very small**. The **negative log-likelihood** helps to **avoid this problem** for many models
- **Rectified linear units** are an **excellent default choice** for **hidden units**
- **Many other types of hidden units are available but are used less frequently.** It can be **difficult to determine when to use which kind** (though rectified linear units are usually an acceptable choice)



Thank you